

Reviewer Pack — Random Matrix Eigenvalue Deficit and SOS Degree for Max-CUT

Entry 7eb129929a17 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 03:53:14 UTC

Random Matrix Eigenvalue Deficit and SOS Degree for Max-CUT

Entry ID: 7eb129929a17 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 03:53:14 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random Max-CUT instance with n variables, the number of eigenvalues of the adjacency matrix lying in $[-\lambda, \lambda]$ is $\Theta(\sqrt{n})$, where $\lambda = 1/\sqrt{n}$. The SOS degree required to approximate Max-CUT with 0.878-accuracy is at least the count of eigenvalues outside $[-\lambda, \lambda]$.

Rationale (proposer’s reasoning):

The spectral properties of the adjacency matrix encode structural constraints on the Max-CUT problem. Eigenvalues outside $[-\lambda, \lambda]$ indicate non-trivial correlations that necessitate higher SOS degrees to capture the problem’s geometry, as per the Goemans-Williamson framework.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREG (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e3a20721267a3b74

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from sys import argv

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        instance = [random.choice([0, 1]) for _ in range(n * (n - 1) // 2)]
        return instance

    def adjacency_matrix(instance, n):
        A = [[0] * n for _ in range(n)]
        idx = 0
        for i in range(n):
            for j in range(i + 1, n):
                A[i][j] = instance[idx]
                A[j][i] = instance[idx]
                idx += 1
        return A

    def power_method(A, max_iter=1000, tol=1e-6):
        n = len(A)
        x = [random.random() for _ in range(n)]
        x /= math.sqrt(sum(x[i]**2 for i in range(n)))

        for _ in range(max_iter):
            y = [sum(A[i][j] * x[j] for j in range(n)) for i in range(n)]
            y_norm = math.sqrt(sum(y[i]**2 for i in range(n)))
            if abs(y_norm - 1) < tol:
                return y
            x = y
        return y

    def eigenvalues(A):
        n = len(A)
        eigvals = []
        for _ in range(30): # Power method to approximate eigenvalues
```

```

        eigval = power_method(A)
        eigvals.append(eigval)
    return eigvals

def sos_degree(n, eigvals):
    lambda_val = 1 / math.sqrt(n)
    count_outside_interval = sum(abs(eigval) > lambda_val for eigval in eigvals)
    return count_outside_interval

n = random.randint(5, 40)
instance = generate_max_cut_instance(n)
A = adjacency_matrix(instance, n)
eigvals = eigenvalues(A)
sos_deg = sos_degree(n, eigvals)

lambda_val = 1 / math.sqrt(n)
count_in_interval = sum(abs(eigval) <= lambda_val for eigval in eigvals)

return {
    "metric_name": "eigenvalue_count",
    "metric_value": count_in_interval,
    "instances_tested": 1,
    "conjecture_holds": sos_deg >= count_outside_interval,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in argv[1:]] if argv[1:] else [random.randint(2, 1000) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5df4622f.py", line 83, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5df4622f.py", line 64, in run_trial
    eigvals = eigenvalues(A)
             ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5df4622f.py", line 52, in eigenvalues
    eigval = power_method(A)
            ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5df4622f.py", line 38, in power_method
    x /= math.sqrt(sum(x[i]**2 for i in range(n)))
TypeError: unsupported operand type(s) for /=: 'list' and 'float'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during eigenvalue computation, preventing data collection | next: Fix the power_method division error and revalidate numerical stability

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	61169
2	propose	ollama_remote	qwen3:8b	0	0	112295
3	propose	ollama_remote	qwen3:8b	0	0	110168
4	preregistration	ollama_remote	qwen3:8b	0	0	24423
5	novelty	ollama_remote	qwen3:8b	0	0	20880
6	novelty	ollama_remote	qwen3:8b	0	0	14718
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14249
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9303
9	judge	ollama_remote	qwen3:8b	0	0	14208

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 381412 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7eb129929a17.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/7eb129929a17.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7eb129929a17.tar.gz` (if generated)